ENTERPRISE COOPERATIVE INFORMATION SYSTEM

Software/System Engineer Step-by-Step Implementation Guide

Project Type: Enterprise Cooperative Management, Banking, Accounting, Loans, Inventory, Merchandising and Regulatory Reporting System

Primary Client: Tauri Desktop Application

Backend: Java + Spring Boot

Database: PostgreSQL

Architecture: Modular Monolith → Evolvable to Microservices

Deployment: Docker / Linux Server

Development Method: Agile, incremental implementation with formal release gates

Purpose

This document is the implementation blueprint for the Software/System Engineer. It defines the recommended implementation sequence, technical foundation, business modules, security controls, financial controls, CIC reporting, testing, deployment, and operational requirements.

1. SYSTEM OBJECTIVE

Develop a unified enterprise system that integrates:

- Cooperative Membership
- Share Capital
- Savings/Deposits
- Loan Management
- Credit Evaluation
- Teller/Cashier Operations
- Accounting/General Ledger
- Treasury and Bank Management
- Procurement
- Inventory/Warehouse
- Merchandising/POS
- Fixed Assets
- CIC Data Preparation and Submission
- Regulatory Reporting
- Document Management
- Workflow and Approval
- Audit Trail
- Notifications
- Management Reporting
- Business Intelligence

The system must operate from a single controlled transaction architecture.

2. NON-NEGOTIABLE SYSTEM PRINCIPLES

2.1 Single Source of Truth

There shall be one authoritative Member Master, one authoritative accounting ledger, one authoritative transaction history, one authoritative inventory ledger, and one authoritative loan subledger.

2.2 No Direct Modification of Posted Financial Transactions

```
POSTED
↓
REVERSAL
↓
CORRECTED TRANSACTION
```

2.3 Double-Entry Accounting

Every applicable financial transaction must generate balanced accounting entries. TOTAL DEBITS = TOTAL CREDITS before posting.

2.4 Maker-Checker Control

```
CREATE → REVIEW → APPROVE → POST
```

2.5 Auditability

- User
- Date/time
- Action
- Entity
- Previous value
- New value
- Transaction/reference number
- IP address where applicable
- Device/session information where appropriate
- Approval information
- Reason for correction/reversal

2.6 Security by Design

Authentication, authorization, encryption, logging, audit, backup, privacy and access controls must be designed into each module.

3. APPROVED TECHNOLOGY STACK

3.1 Desktop Application

- Tauri 2
- React
- TypeScript
- MUI
- TanStack Query
- React Hook Form
- Zod
- Zustand where client-side state is necessary

3.2 Backend Technology

- Java LTS
- Spring Boot
- Spring Security
- Spring Data JPA
- Hibernate
- Flyway
- MapStruct
- Bean Validation
- OpenAPI/Swagger
- JUnit
- Testcontainers

3.3 Database

Use PostgreSQL as the primary system of record. Use foreign keys, constraints, transactions, indexes, check constraints, unique constraints, database views, and appropriate transaction isolation.

3.4 Supporting Technologies

Requirement	Technology
Authentication	Keycloak
Cache	Redis
Event streaming	Apache Kafka
Workflow	Camunda
Documents	MinIO/S3-compatible storage
Reports	JasperReports
BI	Power BI or Metabase
Metrics	Prometheus
Monitoring	Grafana
Logs	Loki
Tracing	OpenTelemetry
Reverse Proxy	Nginx or Traefik
Containers	Docker
CI/CD	GitHub Actions
Security Scanning	Trivy + OWASP tooling

4. IMPLEMENTATION PHASES

```
PHASE 0 Requirements
PHASE 1 Enterprise Architecture
PHASE 2 Development Foundation
PHASE 3 IAM and Security
PHASE 4 Organization
PHASE 5 Member Master
PHASE 6 Accounting Core
PHASE 7 Share Capital
PHASE 8 Savings/Deposits
PHASE 9 Cash/Teller
PHASE 10 Loan Management
PHASE 11 CIC Integration
PHASE 12 Procurement
PHASE 13 Inventory
PHASE 14 Merchandising/POS
PHASE 15 Fixed Assets
PHASE 16 Reporting
PHASE 17 Data Warehouse/BI
PHASE 18 UAT
PHASE 19 Production Deployment
PHASE 20 Operations and Continuous Improvement
```

The engineer must not skip directly to later modules without completing the required foundation.

PHASE 0 — BUSINESS AND SYSTEM REQUIREMENTS

Objective

Understand the cooperative's existing processes before writing software.

Step 0.1 — Identify All Business Units

- Head Office
- Branches
- Satellite offices
- Accounting
- Treasury
- Cashier
- Teller
- Loans
- Membership
- Inventory
- Purchasing
- Merchandising
- HR
- Management
- Internal Audit
- Compliance
- DPO
- CIC reporting

Step 0.2 — Document Existing Processes

```
CURRENT PROCESS
↓
PROBLEMS
↓
REQUIRED CONTROLS
↓
TARGET PROCESS
```

Document membership registration, share capital payment, savings deposit/withdrawal, loan application/approval/release/payment, delinquency, cash count, purchasing, receiving, inventory, POS, accounting closing, and CIC submission.

Step 0.3 — Identify Business Rules

- Loan policies
- Interest rates
- Penalty rules
- Membership requirements
- Share capital rules
- Savings rules
- Approval limits

- Cash limits
- Inventory rules
- Discount rules
- Accounting policies
- Closing procedures
- Regulatory requirements

Step 0.4 — Produce BRD

- Approved Business Requirements Document
- Process maps
- Business rules
- Regulatory requirements matrix
- User-role matrix

Acceptance: management confirms the documented processes accurately represent requirements.

PHASE 1 — ENTERPRISE ARCHITECTURE

Objective

Design the system before development.

Step 1.1 — System Context

```
Cooperative System
■
■■■ CIC
■■■ Banks
■■■ Email/SMS
■■■ Payment Providers
■■■ Document Storage
■■■ BI
```

Step 1.2 — C4 Architecture

- Context Diagram
- Container Diagram
- Component Diagram
- Deployment Diagram

Step 1.3 — System Boundaries

```
organization
security
membership
share
savings
loan
cash
accounting
treasury
procurement
inventory
sales
fixedasset
document
workflow
cic
notification
audit
reporting
```

Step 1.4 — Architecture Decision Records

```
ADR-001 Tauri Desktop
ADR-002 Spring Boot
ADR-003 PostgreSQL
ADR-004 Modular Monolith
ADR-005 Keycloak
ADR-006 Kafka
ADR-007 Redis
ADR-008 MinIO
ADR-009 Flyway
```

Deliverables: system architecture, C4 diagrams, technology architecture, deployment architecture, security architecture, integration architecture, and ADRs.

PHASE 2 — PROJECT FOUNDATION

Step 2.1 — Repository

```
coop-platform/
■■■ apps/
■ ■■■ desktop/
■ ■■■ admin-web/
■■■ backend/
■ ■■■ coop-core/
■■■ infrastructure/
■ ■■■ docker/
■ ■■■ postgres/
■ ■■■ keycloak/
■ ■■■ redis/
■ ■■■ kafka/
■ ■■■ minio/
■■■ database/
■ ■■■ migrations/
■ ■■■ seed/
■■■ documentation/
■■■ scripts/
■■■ .github/
■■■ workflows/
```

Step 2.2 — Docker Development Environment

```
postgres
redis
kafka
keycloak
minio
grafana
prometheus
loki
```

The developer must be able to start the development environment with a single command such as docker compose up -d.

Step 2.3 — Git Strategy

```
main
develop
feature/*
release/*
hotfix/*
```

Step 2.4 — CI/CD

```
Commit → Build → Unit Test → Integration Test → Security Scan → Docker Build → Deploy Staging → UAT
→ Production Approval → Production
```

PHASE 3 — IDENTITY AND ACCESS MANAGEMENT

Step 3.1 — Configure Keycloak

- Users
- Groups
- Roles
- Permissions
- MFA
- Password policies
- Session management
- OAuth2/OIDC

Step 3.2 — Define Roles

```
SYSTEM_ADMIN
CEO
GENERAL_MANAGER
ACCOUNTING_HEAD
ACCOUNTANT
BOOKKEEPER
TREASURER
HEAD_CASHIER
TELLER
LOAN_OFFICER
LOAN_PROCESSOR
LOAN_APPROVER
CREDIT_COMMITTEE
MEMBERSHIP_OFFICER
INVENTORY_HEAD
WAREHOUSE_CLERK
PURCHASING_OFFICER
AUDITOR
DPO
CIC_OPERATOR
CIC_COMPLIANCE_OFFICER
```

Step 3.3 — Permission Model

```
loan.application.create
loan.application.review
loan.application.approve
loan.release
loan.payment.post
accounting.journal.create
accounting.journal.approve
accounting.period.close
member.view
member.create
member.update
cic.prepare
cic.validate
cic.submit
cic.correct
```

Step 3.4 — Segregation of Duties

```
Teller ≠ Teller Supervisor
Loan Processor ≠ Loan Approver
Journal Creator ≠ Journal Approver
```

PHASE 4 — ORGANIZATION MANAGEMENT

```
organization
branch
office
department
business_unit
cost_center
fiscal_year
accounting_period
```

Support: Cooperative → Branch → Department → Employee/User.

PHASE 5 — MEMBER MASTER

Objective

Create the central member identity.

Step 5.1 — Member Registration

```
member
membership_application
membership_status_history
```

Step 5.2 — Member Information

- Full name
- Birth information
- Address
- Contact information
- Employment
- Identification
- KYC information
- Beneficiaries
- Membership status

Step 5.3 — Document Management

Store documents in MinIO/S3-compatible storage and metadata in PostgreSQL.

Step 5.4 — Member Number

```
MEM-0000001234
```

Acceptance: a member can proceed from Application → Approval → Member Creation → Share Account without creating duplicate member records.

PHASE 6 — ACCOUNTING CORE

This phase must be completed before deep financial integration.

Step 6.1 — Chart of Accounts

Configure the applicable cooperative chart of accounts based on current CDA requirements adopted by the cooperative. Do not hard-code the chart of accounts into Java source code.

Step 6.2 — Accounting Period

```
OPEN
SOFT CLOSED
CLOSED
LOCKED
```

Step 6.3 — General Ledger

```
journal
journal_line
ledger
account_balance
```

Step 6.4 — Double-Entry Validation

```
Debit = Credit
If false → REJECT TRANSACTION
```

Step 6.5 — Journal Posting

```
DRAFT → FOR REVIEW → APPROVED → POSTED
```

Step 6.6 — Reversal

```
Posted Transaction → Reversal → Correct Transaction
```

Step 6.7 — Financial Statements

- Statement of Financial Condition
- Statement of Operations
- Statement of Changes in Equity
- Statement of Cash Flows
- Supporting schedules

Step 6.8 — Trial Balance

The system must generate the trial balance automatically.

Acceptance: every posted journal demonstrates Debits = Credits.

PHASE 7 — SHARE CAPITAL

```
share_product
share_account
share_subscription
share_transaction
share_certificate
share_transfer
share_withdrawal
```

```
Subscription → Approval → Payment → Accounting
```

PHASE 8 — SAVINGS / DEPOSITS

```
deposit_product
deposit_account
deposit_transaction
deposit_interest
deposit_rate
deposit_hold
```

Transactions: Deposit, Withdrawal, Transfer, Interest, Adjustment, Closure. Each transaction must integrate with accounting.

PHASE 9 — CASH AND TELLER

```
cash_drawer
teller
cash_transaction
cash_transfer
cash_count
cash_variance
vault
bank_account
bank_reconciliation
```

```
Login → Open Cash Drawer → Process Transactions → Supervisor Verification → End-of-Day Count →
Variance Check → Close Drawer
```

PHASE 10 — LOAN MANAGEMENT

Step 10.1 — Loan Products

```
loan_product
```

Parameters: interest method, interest rate, term, frequency, penalty, fees, grace period, amortization method, approval limits.

Step 10.2 — Loan Application

```
loan_application
loan_borrower
loan_co_borrower
loan_guarantor
loan_collateral
```

Step 10.3 — Credit Evaluation

Record income, expenses, existing obligations, payment capacity, collateral, member history, loan exposure and applicable credit-history information.

Step 10.4 — Approval

```
Loan Amount → Required Approval Level → Approver → Decision
```

Step 10.5 — Loan Booking

```
Application → Approved → Booked → Active
```

Step 10.6 — Loan Release

```
Loan Receivable → Accounting → Cash/Bank
```

Step 10.7 — Amortization

Build a configurable amortization engine supporting the cooperative's approved products.

Step 10.8 — Loan Payment

```
Payment → Penalty → Fees → Interest → Principal
```

Step 10.9 — Delinquency

```
Current
Past Due
Delinquent
Restructured
Written Off
Recovered
Closed
```

Step 10.10 — Loan Restructuring

Maintain the complete history. Never overwrite original loan history.

PHASE 11 — CIC DATA SUBMISSION

Objective

Create a controlled CIC regulatory reporting subsystem.

Step 11.1 — CIC Data Model

```
cic_submission
cic_submission_batch
cic_submission_record
cic_validation_error
cic_response
cic_correction
```

Step 11.2 — Data Mapping

Map internal Member, Loan, Payment, Balance, Status and Identification data to the applicable CIC submission structure. Version the mapping so specification changes can be implemented without rewriting the loan system.

Step 11.3 — Validation

```
VALID
WARNING
ERROR
```

Step 11.4 — Submission Workflow

```
Generate → Validate → Review → Approve → Submit → Receive Result → Reconcile → Correct Errors
```

Step 11.5 — CIC Audit

- Submission period
- User
- Date
- Batch

- Number of records
- Accepted
- Rejected
- Errors
- Corrections
- Final status

PHASE 12 — PROCUREMENT

```
supplier
purchase_request
purchase_order
purchase_order_line
goods_receipt
supplier_invoice
accounts_payable
supplier_payment
```

```
Purchase Request → Approval → Purchase Order → Receiving → Invoice → Payable → Payment → Accounting
```

PHASE 13 — INVENTORY

```
product
product_category
unit_of_measure
warehouse
warehouse_location
inventory_balance
inventory_transaction
inventory_batch
stock_count
stock_adjustment
```

Support RECEIVE, ISSUE, SALE, RETURN, TRANSFER, ADJUSTMENT, DAMAGE and EXPIRY. Do not simply modify quantity fields without recording the transaction.

PHASE 14 — MERCHANDISING / POS

```
product catalog
pricing
discount
promotion
sales
sales lines
returns
payments
receipts
```

```
Sale → Payment → Inventory Reduction → Revenue → Cost of Goods Sold → Accounting
```

PHASE 15 — FIXED ASSETS

```
asset
asset_category
asset_acquisition
asset_transfer
asset_depreciation
asset_maintenance
asset_disposal
```

Support depreciation schedules and accounting integration.

PHASE 16 — DOCUMENT MANAGEMENT

```
document_id
entity_type
entity_id
object_key
file_name
mime_type
hash
classification
created_by
created_at
retention_date
```

Documents should not be freely downloadable by every authenticated user. Access must be permission-controlled.

PHASE 17 — WORKFLOW ENGINE

Implement configurable workflows for Membership, Loan, Purchasing, Journal Approval, Cash Adjustments, Inventory Adjustments and CIC Submission.

```
LOAN
Officer
↓
Supervisor
↓
Credit Committee
↓
Authorized Approver
↓
Release
```

Approval limits must be configurable.

PHASE 18 — AUDIT AND COMPLIANCE

Implement centralized audit logging. Every significant transaction must record WHO, WHAT, WHEN, WHERE, WHY, BEFORE and AFTER. Audit logs must be protected against ordinary users modifying or deleting them.

PHASE 19 — DATA PRIVACY

```
Data classification
Privacy notices
Access controls
Retention rules
Data subject request tracking
Data sharing records
DPIA references
Incident management
Data export controls
Data deletion/retention workflows
```

Protect government IDs, financial information, loan records, CIC information, sensitive member information, documents and authentication information. Final configuration must be reviewed against applicable Philippine privacy requirements before production.

PHASE 20 — REPORTING

Membership

- New Members

- Active Members
- Inactive Members
- Membership Growth

Loans

- Loan Portfolio
- PAR
- Delinquency
- Collection
- Maturity
- Outstanding Balance
- Loan Releases
- Loan Payments

Accounting

- Trial Balance
- General Ledger
- Balance Sheet
- Income/Operations
- Cash Flow

Inventory

- Stock Balance
- Movement
- Low Stock
- Stock Valuation
- Stock Count Variance

POS

- Daily Sales
- Sales by Product
- Sales by Branch
- Returns
- Discounts

CIC

- Submission Status
- Accepted
- Rejected
- Errors
- Corrections

PHASE 21 — MANAGEMENT DASHBOARD

Board / Management

```
Members
Share Capital
Savings
Loan Portfolio
PAR
Collection
Cash
Inventory
Revenue
Expenses
Net Surplus
```

Operations

```
Pending Approvals
Teller Transactions
Loan Applications
Delinquencies
Inventory Alerts
CIC Errors
```

PHASE 22 — DATA WAREHOUSE

```
PostgreSQL → ETL/ELT → Data Warehouse → BI
```

After the transactional system stabilizes, move complex executive analytics to a reporting/data-warehouse layer rather than running all analytics against heavily used transaction tables indefinitely.

PHASE 23 — TESTING

Unit Testing

- Interest calculation
- Amortization
- Penalties
- Accounting rules
- Inventory calculations
- Permissions

Integration Testing

```
Loan → Accounting
Savings → Accounting
POS → Inventory
POS → Accounting
Payment → Loan
Payment → Accounting
CIC → Loan
```

End-to-End Testing

```
Member Registration → Share Payment → Savings Account → Loan Application → Credit Evaluation →
Approval → Loan Release → Payment → Accounting → CIC
```

PHASE 24 — RECONCILIATION TESTING

```
Loan Subledger = Loan GL Balance
Savings Subledger = Savings GL Balance
Inventory = Inventory GL Balance
Cash/Teller = Cash GL Balance
```

Any difference must generate an exception.

PHASE 25 — SECURITY TESTING

- Authentication testing
- Authorization testing
- Privilege escalation testing
- API security testing
- SQL injection testing
- XSS testing
- CSRF testing where applicable
- File upload testing
- Session testing
- Password testing
- Rate-limit testing
- Dependency scanning
- Container scanning
- Secrets scanning

PHASE 26 — PERFORMANCE TESTING

Test concurrent users, concurrent tellers, loan transactions, POS transactions, report generation, database load and CIC batch processing using realistic production-like datasets.

PHASE 27 — USER ACCEPTANCE TESTING

```
ACCOUNTING UAT
1. Create journal
2. Approve journal
3. Post journal
4. Reverse journal
5. Generate trial balance
6. Generate financial statements
```

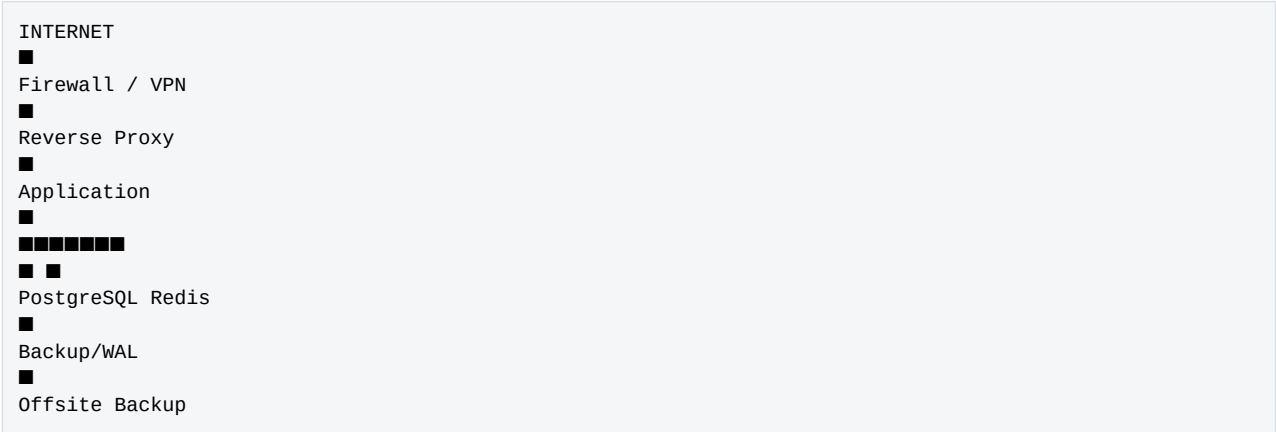
Repeat UAT for Membership, Loans, Teller, Treasury, Inventory, Purchasing, POS and CIC.

PHASE 28 — DATA MIGRATION

```
Existing System → Data Extraction → Data Cleaning → Mapping → Transformation → Validation → Test Migration → Reconciliation → Final Migration
```

Never migrate directly into production without a test migration.

PHASE 29 — PRODUCTION INFRASTRUCTURE



For larger deployments, use a load balancer with multiple application instances and a properly designed PostgreSQL high-availability strategy.

PHASE 30 — BACKUP AND DISASTER RECOVERY

- Daily Full Backup
- WAL/PITR
- Offsite Backup
- Encrypted Backup
- Backup Monitoring
- Restore Testing

Define RPO (Recovery Point Objective) and RTO (Recovery Time Objective). Example targets such as RPO ≤ 5 minutes and RTO ≤ 1 hour must be approved by management based on operational and budget requirements.

PHASE 31 — PRODUCTION DEPLOYMENT

- Security approved
- Privacy review completed
- Backup tested
- Restore tested
- UAT signed
- Data migration tested
- Accounting reconciled
- Loan balances reconciled
- Savings reconciled
- Inventory reconciled
- CIC process tested
- User accounts created
- Roles reviewed
- Documentation completed
- Training completed

• Disaster recovery tested

PHASE 32 — GO-LIVE

```
Pilot → Limited Branch → Stabilization → Additional Branches → Full Deployment
```

Avoid deploying the entire organization simultaneously unless adequate testing and rollback capability exists.

PHASE 33 — POST-GO-LIVE

For the first 30–90 days monitor transaction failures, accounting discrepancies, user errors, security events, database performance, API performance, CIC errors, backup status and audit logs. Maintain a formal issue-management process.

34. DATABASE MODULE STRUCTURE

```
coop
organization
security
membership
share
savings
loan
cash
accounting
treasury
procurement
inventory
sales
fixedasset
document
workflow
cic
notification
audit
reporting
```

35. CORE RELATIONSHIP

```
MEMBER
■
■■■ Share Account
■■■ Savings Accounts
■■■ Loans
■■■ Transactions
■■■ Documents
■■■ Beneficiaries
```

```
BUSINESS TRANSACTION
■
▼
SUBLEDGER
■
▼
ACCOUNTING
■
▼
GENERAL LEDGER
■
▼
FINANCIAL REPORTING
```

36. FINANCIAL TRANSACTION CONTROL

Every financial transaction must have a unique business reference.

```
DEP-2026-00000001
WDL-2026-00000001
LOAN-2026-00000001
PAY-2026-00000001
JNL-2026-00000001
POS-2026-00000001
```

Never rely on database IDs alone as the business transaction reference.

37. TRANSACTION STATUS MODEL

```
DRAFT
PENDING
FOR_REVIEW
APPROVED
REJECTED
POSTED
REVERSED
CANCELLED
```

Do not allow arbitrary status strings.

38. API STANDARDS

```
/api/v1/members
/api/v1/loans
/api/v1/loans/{id}
/api/v1/loans/{id}/payments
/api/v1/accounts
/api/v1/tellers
/api/v1/inventory
/api/v1/sales
/api/v1/cic/submissions
```

All APIs must have authentication, authorization, validation, error handling, logging, audit where required, and API versioning.

39. ERROR HANDLING

Never return raw Java exceptions to users.

```
{"code":"LOAN-001","message":"Loan cannot be released.","details":"Required approval has not been
completed.","reference":"ERR-2026-000123"}
```

The reference number should appear in system logs.

40. DOCUMENTATION REQUIRED

- System Architecture Document
- Database Design Document
- API Documentation
- Security Architecture
- Deployment Guide
- Backup Guide
- Disaster Recovery Guide

- User Manual
- Administrator Manual
- Developer Manual
- Data Dictionary
- CIC Integration Guide
- Accounting Configuration Guide
- Release Notes
- Change Log

41. DEVELOPMENT RULE

```
React Component
↓
API
↓
Application Service
↓
Domain Logic
↓
Repository
↓
PostgreSQL
```

Do not implement core business logic directly inside UI components.

42. FINANCIAL RULE

```
Tauri
↓
HTTPS
↓
Spring Boot
↓
Domain Service
↓
PostgreSQL
```

Never allow the Tauri application to connect directly to PostgreSQL.

43. VERSION CONTROL RULE

Every database change must use a migration.

```
V001__create_organization.sql
V002__create_users.sql
V003__create_members.sql
V004__create_share_accounts.sql
V005__create_chart_of_accounts.sql
```

Never manually modify production tables without a controlled migration.

44. RELEASE MANAGEMENT

- Version
- Release date
- Changes

- Database migrations
- Known issues
- Rollback procedure
- Security changes
- Affected modules
- Approval

```
v1.0.0
v1.1.0
v1.2.0
v1.2.1
```

45. DEFINITION OF DONE

- Database complete
- API complete
- Business logic complete
- Authorization complete
- Audit complete
- Validation complete
- Error handling complete
- Accounting integration complete where applicable
- Reports complete
- Unit tests complete
- Integration tests complete
- UAT complete
- Documentation complete
- Security review complete

46. FIRST PRODUCTION RELEASE

```
1. Organization
2. IAM/Security
3. Membership
4. Share Capital
5. Accounting
6. Savings
7. Teller/Cash
8. Basic Loan Management
9. Audit
10. Reporting
```

Then progressively add CIC, Procurement, Inventory, POS, Fixed Assets and BI.

47. DEVELOPMENT PRIORITY



Accounting should be designed early and progressively integrated with each transaction domain.

48. PROJECT GOVERNANCE

```
Management
IT
Accounting
Treasury
Loans
Membership
Inventory
Internal Audit
Compliance
DPO
CIC Officer
End Users
```

No major financial business rule should be implemented solely based on developer assumptions.

49. CHANGE CONTROL

Any change affecting Accounting, Loan computation, Interest, Penalties, Share Capital, Savings, CIC, Privacy, User permissions or Financial reports must undergo formal change review.

50. FINAL IMPLEMENTATION PHILOSOPHY

```
USER
↓
TAURI
↓
API
↓
APPLICATION SERVICE
↓
DOMAIN RULES
↓
APPROVAL / CONTROL
↓
TRANSACTION
↓
SUBLEDGER
↓
GENERAL LEDGER
↓
AUDIT
↓
REPORTING
↓
REGULATORY OUTPUT
```

The system should be built from the transaction and accounting architecture outward, rather than from the user interface inward.

51. ENGINEER'S FIRST 30 DAYS

Week 1

- Requirements gathering
- Existing system assessment
- Business process interviews
- Regulatory requirements inventory

Week 2

- System architecture
- C4 diagrams
- Database architecture
- Security architecture
- Network architecture

Week 3

- Repository
- Docker environment
- PostgreSQL
- Spring Boot
- Tauri
- Keycloak
- Redis
- CI/CD

Week 4

- IAM
- Organization
- Audit framework
- Database migration framework
- API framework
- Base UI framework

At the end of Day 30, the system should have a functioning technical foundation, even though the major business modules are not yet complete.

52. FIRST 90-DAY TARGET

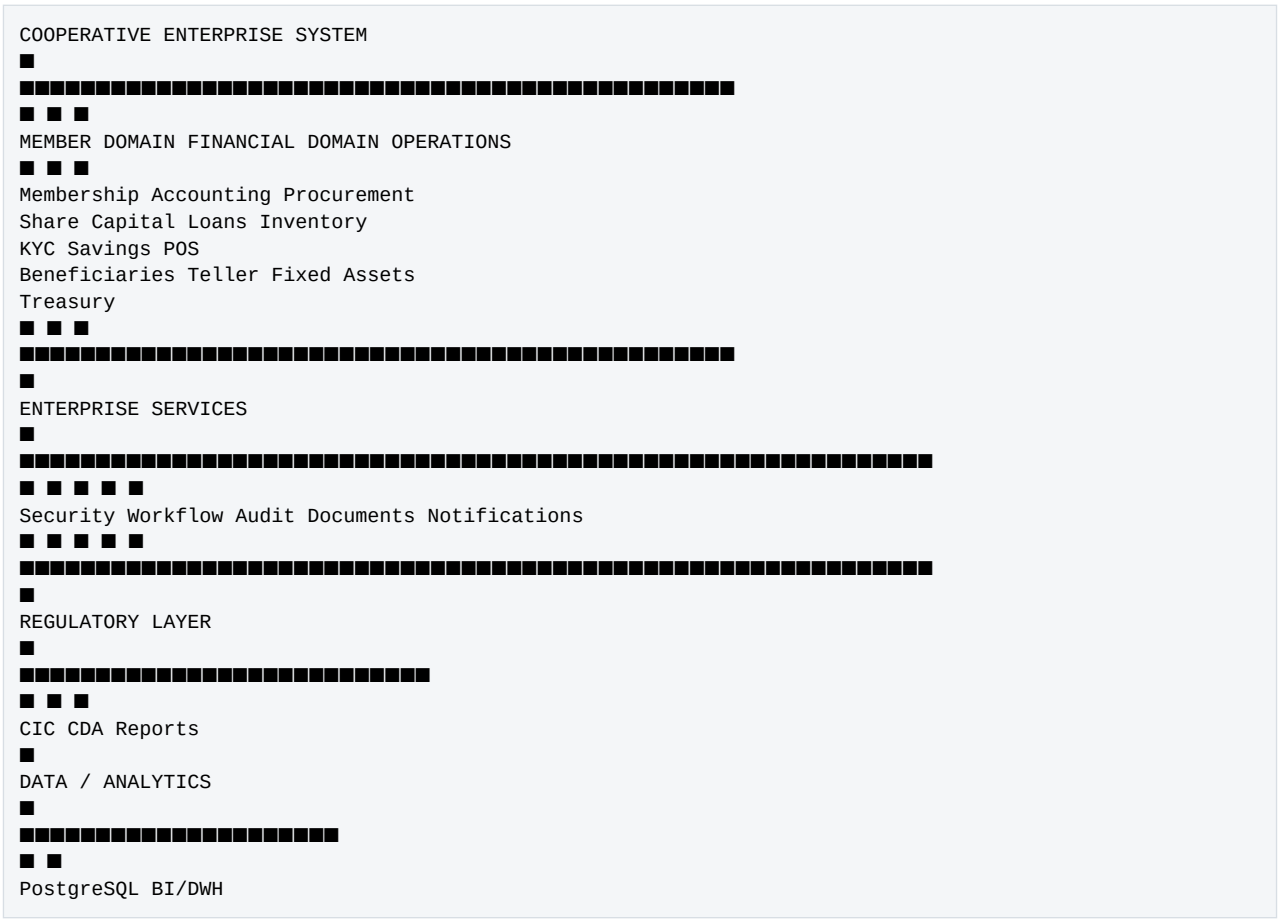
```
✓ Authentication
✓ Authorization
✓ Organization
✓ User management
✓ Audit
✓ Member Master
✓ Membership application
✓ Share Capital
✓ Basic Accounting
✓ Chart of Accounts
✓ General Ledger
✓ Trial Balance
✓ Basic Reports
```

53. SUCCESS CRITERIA FOR THE ENTIRE PROJECT

- Financial transactions are traceable
- Accounting balances automatically
- Loan subledger reconciles with GL
- Savings subledger reconciles with GL
- Inventory reconciles with GL
- Teller balances reconcile
- Users have controlled permissions
- Maker-checker controls operate
- Posted transactions cannot be silently changed
- Complete audit trail exists
- CIC submissions are controlled and traceable
- Reports are reproducible
- Backups are tested
- Disaster recovery is tested
- Security testing is completed
- Privacy controls are implemented
- UAT is signed off
- Documentation is complete

• Production monitoring is operational

54. FINAL TARGET ARCHITECTURE



ENGINEERING HAND-OFF CHECKLIST

- Confirm this implementation guide is approved as the engineering baseline.
- Complete Phase 0 requirements and process mapping before major feature development.
- Create the architecture, database, security and integration design documents.
- Establish the development environment and CI/CD pipeline.
- Implement IAM, organization, audit and migration foundations first.
- Build Member Master and Accounting Core before deep financial integrations.
- Require reconciliation tests for every financial subledger.
- Treat CIC as a controlled regulatory subsystem with versioned mappings and validation.
- Perform security, privacy, UAT, backup/restore and disaster-recovery validation before production.